ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS
DEPARTMENT OF INFORMATICS

Computer Programming with Java Assignment

ATHANASIOS GOURDOMICHALIS

PART 2

Assignment Purpose

The objective of the second part is to process data from a file and develop mechanisms for statistical analysis and movie recommendation. You are called to extend your object-oriented model, utilize Java Collections and grouping techniques, and implement basic forms of **collaborative filtering** (if two users share similar tastes, movies liked by one may also appeal to the other) and **content-based filtering** (if a user likes movies with specific characteristics, we recommend other movies with similar attributes) to support recommendation features. The program must handle **exceptions** appropriately.

Available File: reviews.csv

Contains rows with information regarding:

- **Movie:** title, release year, genre
- **User:** unique identifier or username
- **Review:** numerical rating (e.g., 1–10) and comment

Context

Now that the platform possesses the foundational infrastructure for representing movies, users, and reviews, it seeks to leverage real-world data to extract statistical insights, understand user preferences, and enhance its recommendation mechanisms. Users continue to rate movies using numerical scores and short comments, and the company aims to organize and analyze this information on a larger scale.

Description

Key questions concerning the platform include:

- **Which are the most popular movies per genre based on user ratings?**
- **How can the movies with the highest user acceptance be identified?**
- **What are the preferences of each individual user?**
- **Which movies are worth recommending to a user based on their previous ratings or the ratings of others with a similar profile?**

To support these objectives, you are required to extend the object-oriented system to:

- **Read data directly from files,**
- **Organize and group movies and users based on ratings,**
- **Extract useful statistical analytics,**
- **Support recommendation mechanisms based on user similarity (collaborative filtering) and movie content (content-based filtering).**

Required Steps

1. File Parsing and Object Instantiation

- **Read the input file:** Parse the reviews.csv dataset.
- **Instantiate core domain objects:**
  - Movie (title, release year, genre, list of reviews)
  - User (username/ID, list of reviews)
  - Review (movie, user, rating, comment)

2. Data Organization and Processing

Organize movies:

- **By genre:**

- Utilize a **`Map<String, List<Movie>>`**, where the key String represents the genre (e.g., "Action") and the value `List<Movie>` contains the corresponding movies.
- If a movie belongs to multiple genres (e.g., "Action|Sci-Fi"), you may assume it belongs to a single genre. Alternatively, you may split the string (e.g., `split("|")`) and insert the movie into each respective genre list, ensuring it is not double-counted in aggregate statistics (e.g., total movie count).

- **Sort by average rating and review volume:**

- Sort each List<Movie> using a custom Comparator<Movie> based on `getAverageRating()` or `getReviewCount()`.
- Optional: You may use a `TreeMap` or `PriorityQueue` if automatic order preservation is preferred.

3. Statistical Analytics & Queries

- **Top-5 movies per genre** ordered by average rating.
- **Average user rating** across submitted reviews.
- **High-appraisal movies:** Movies rated above 7.0 by at least 80% of the users who reviewed them.

4. Implementation of the Recommender System

Implement two basic recommendation algorithms:

a. User-Based Collaborative Filtering:

- Calculate user similarity mechanisms (e.g., correlation coefficient or mean absolute error of ratings on shared movies).
- Identify the most similar users (neighbors) and recommend highly-rated movies from those users that the target user has not yet rated.

b. Content-Based Filtering:

- Analyze the genres and ratings of movies previously evaluated by the target user.
- Recommend unseen movies belonging to genres the user prefers, calculated using their average rating per genre.

Hints & Recommendations

- **Genre Grouping:** You may utilize a `Map<String, List<Movie>>` to handle grouping by genre.
- **Top-N Sorting:** To retrieve the Top-5 entries, you can employ a custom `Comparator` along with `Collections.sort()`.
- **User Similarity Metric:** User similarity can be computed using the **Mean Squared Error (MSE)** of their shared movie ratings. A lower MSE value indicates higher similarity between user ratings. In the specific case of an MSE $= 0$, both users have assigned identical ratings across all shared movies. You may classify two users as having similar preferences if their MSE $<$ 4.0.

Below is an example with two users and the calculation of the MSE:

Movie	User A	User B
The Matrix	9	10
La La Land	7	6
Joker	8	7

$$\text{MSE} = \frac{(9-10)^2 + (7-6)^2 + (8-7)^2}{3} = \frac{1+1+1}{3} = \mathbf{1.0}$$

$\Longrightarrow$ the users have similar preferences, since MSE < 4.

- You may create a `Recommender` class containing the two recommendation methods (collaborative and content-based filtering; **see also the source code draft**).
- If you use collection structures such as `HashSet<Movie>` or `Map<Movie, ...>`, you must implement the `equals()` and `hashCode()` methods within the Movie class. Movie equality comparison can be based on **title and release year**, e.g.,

```
1   @Override
2   public boolean equals(Object obj) {
3       if (this == obj) return true;
4       if (!(obj instanceof Movie)) return false;
5       Movie other = (Movie) obj;
6       return this.title.equals(other.title) && this.year == other.year;
7   }
8
9   @Override
10  public int hashCode() {
11      return Objects.hash(title, year);
12  }
13
```

Assignment Instructions

Submission Requirements:

1. **The assignment must be submitted as a Java project containing**:

   1.1. **Documented Source Code**: Well-commented source code.

   1.2. **`README` File**: A README file detailing the system architecture/design, features, and instructions for execution.

   1.3. A **demo** in **`Main.java`** file showcasing representative execution examples (you may choose how the main method operates).

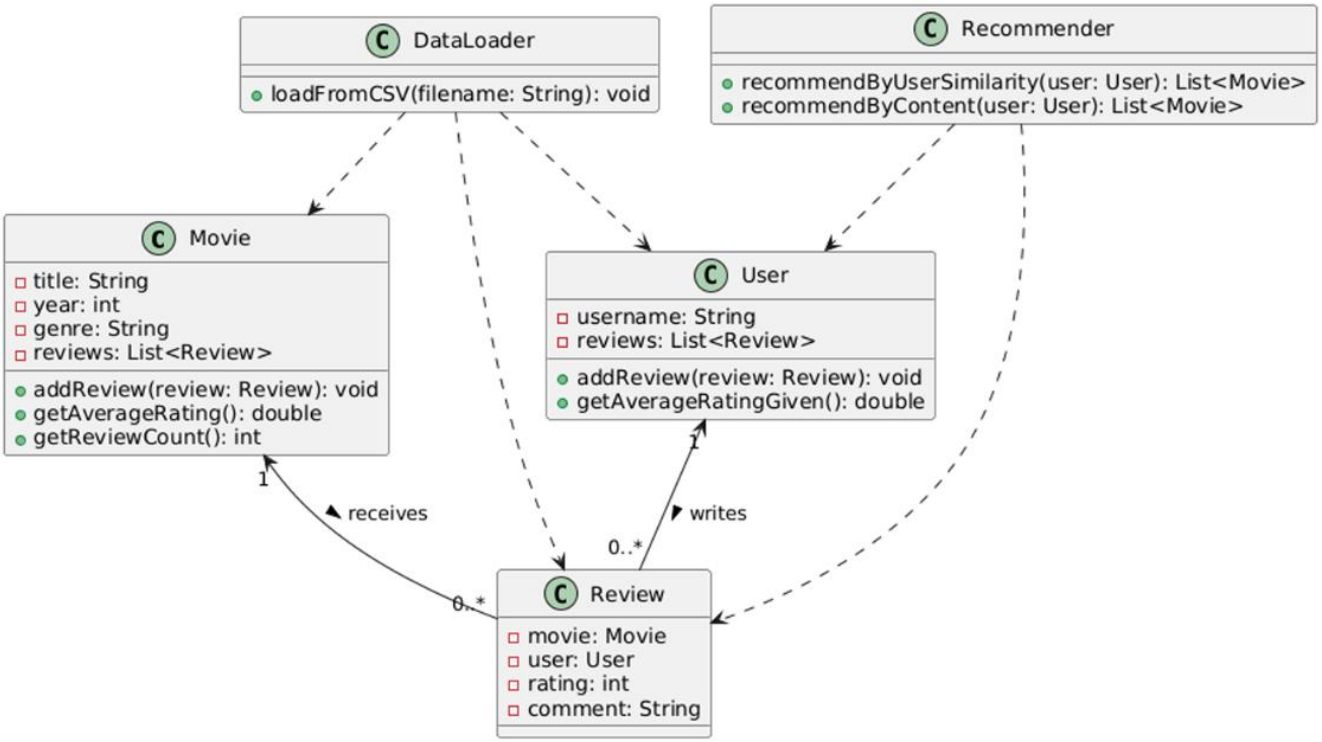
2. **Implementation Guidelines:** During the implementation of our assignment, we have to adhere to these following rules:

   2.1. Entry Point: Name the class containing the main method **Main** and its corresponding file **Main.java**. Do not forget to include the personal details (names/IDs) of all team members as a comment at the top of this file.

   2.2. Package Structure: Do not group your classes **into Java packages** (keep all classes in the default package).

   2.3. Compilation & Execution: The submitted code must compile and run directly from the **command line** without relying on any Integrated Development Environment (IDE).

Draft UML Diagram *(subject to change)*


DataLoader
+ loadFromCSV(filename: String): void
Recommender
+ recommendByUserSimilarity(user: User): List<Movie>
+ recommendByContent(user: User): List<Movie>
Movie
- title: String
- year: int
- genre: String
- reviews: List<Review>
+ addReview(review: Review): void
+ getAverageRating(): double
+ getReviewCount(): int
User
- username: String
- reviews: List<Review>
+ addReview(review: Review): void
+ getAverageRatingGiven(): double
1
receives
0..*
1
writes
0..*
Review
- movie: Movie
- user: User
- rating: int
- comment: String